

Pseudo-Oracle Conformance Testing and Pre-Registered Held-Out Mutation Analysis for a Privacy-Critical Secure-Computation Application

Anonymous Author(s)

Paper submitted for double-blind review

Artifact (checker, oracle, mutants, evidence) provided as anonymized supplementary material

Abstract—Continuous integration turns a security property into a green check, and a green check is easily mistaken for a guarantee. We report a reproducible *case study in testing* that green check for one secure multi-party computation (MPC) application: the deterministic-query fragment of the knowledge-threshold belief computation of Mardziel, Hicks, Katz and Srivatsa (PLAS 2012), implemented on MP-SPDZ with three-party replicated secret sharing. The subject has no ground-truth oracle, so we build a separately implemented plaintext *pseudo-oracle*, an *anti-echo* interface that keeps expected outputs out of the system under test, a strict runtime transcript check, and recomputed, hash-bound test evidence; a bounded 228-case adversarial matrix and five committed executable negative controls exercise a static *delivery-conformance* checker that inspects compiled assembly. We evaluate that checker’s adequacy by *mutation analysis* under a build-time fault model, and—because an adaptive mutation sequence overfits the tool it hardens—we add a *pre-registered, out-of-sample held-out mutation study*: 22 mutants with predictions committed *before* execution and run once against the frozen checker. The decisive result is a *surviving, source-realizable* mutant, H-R2: a multisets-preserving mis-delivery that the structural checker accepts and that only a downstream transcript oracle catches, for that concrete mutant. We report exact-strength results and honest mutant accounting (killed / survived / stillborn), and we deliberately avoid an aggregate mutation score for manifest-level probes whose source realizability is unproven. The whole study reproduces from a fresh checkout, and the compiled-assembly signatures reproduce on the pinned backend and one additional MP-SPDZ version. We do not claim a new analysis technique or a privacy proof; the contribution is a conformance-testing and mutation-adequacy methodology, applied and reproduced, and an honest map of what it establishes and where it stops.

Index Terms—software testing, mutation testing, mutation adequacy, test oracles, pseudo-oracle, conformance testing, pre-registration, reproducibility, negative results, secure multi-party computation.

I. INTRODUCTION

An MPC framework such as MP-SPDZ [2] compiles a program into a protocol that several parties execute so each learns only its prescribed output. Building a correct *application* on top of such a framework is a separate task from the framework’s own security argument: an application author can, by accident or by an injected fault, write source that reveals a value the functionality was meant to protect. When such an application is wired into continuous integration (CI), a passing test suite is easy to mistake for evidence that the privacy property holds. This paper is a case study of *what that*

test suite can and cannot establish, told in the vocabulary of software testing: test oracles, conformance tests, and mutation adequacy.

We are explicit about what this paper is *not*. It is not “the first implementation of PLAS belief tracking,” and makes no such claim. It is not a new program-analysis *technique*: the observation that information-flow safety depends on dataflow and semantics rather than instruction names is prior art (§IX). We frame the work as a *testing case study / experience report* whose value is (i) a concrete conformance-test design for a subject with no ground-truth oracle, (ii) a *pre-registered, held-out* mutation study of a static conformance checker, and (iii) an honest, reproducible account of test adequacy—including a surviving mutant that reveals a real fault the checker misses.

a) *The testing problem*: The subject computes, per recipient, whether releasing a query’s output would let a protected party’s posterior belief exceed a public threshold, and—on accept—delivers the output *privately* to only the intended recipient. Two properties must be tested: functional correctness of the accept/reject/mask decision, and the *delivery* discipline (only party j learns $(\text{accept}_j, \text{payload}_j)$). There is no reference implementation to serve as an oracle, and the naive way to obtain one—feeding expected outputs into the harness—risks the classic *oracle-in-the-SUT* anti-pattern, where the test passes because the expected answer leaked into the system under test.

b) *Contributions (all testing, no new research)*:

- 1) A **conformance-test design** for a privacy-critical MPC application (§III): a separately implemented plaintext *pseudo-oracle*, an *anti-echo* interface that keeps expected outputs and post-state out of the system under test, a strict runtime transcript check that fails closed on the enumerated cases, and recomputed, hash-bound test evidence.
- 2) A **reproducible test artifact** (§III-A): a fixture, a bounded 228-case adversarial matrix checked against the pseudo-oracle, and five committed executable negative controls, run against a pinned backend.
- 3) A **mutation analysis** of a static delivery-conformance checker (§IV), with honest *killed / survived / stillborn* accounting, and—to counter the overfitting inherent in an adaptive mutation sequence—a **pre-registered, out-of-sample held-out mutation study** (§V) whose decisive

result is a *surviving* source-realizable mutant (H-R2).

- 4) A statement of **test adequacy and its boundary** (§VII): what the layered checks establish operationally, and the two verification obligations a privacy *guarantee* would additionally need.

We also report the reproduction of the whole study from a fresh checkout and a cross-version replication of the backend-level test observations (§VI). Every number, mutant, and result below is unchanged from a frozen artifact baseline; this paper re-frames and re-packages that testing evidence.

II. SUBJECT AND FAULT MODELS

a) The subject under test (deterministic-query fragment):

Following PLAS 2012 [1], N parties hold secrets in a finite domain D ; each observer j holds a belief over the others' secrets. For a query Q , the mechanism checks, per recipient, whether releasing Q 's output would let any protected party's posterior exceed a public threshold, quantifying over *every possible output* so the accept/reject decision is simulatable; on accept the recipient privately learns the output, on reject a masked constant, and no party may learn another's verdict. We test deterministic, total, public queries; two instantiate the subject: `pl_is_max` and `sum_even`.

b) *Backend*: Circuits are compiled and run with MP-SPDZ pinned at 9d809599, backend replicated-ring-party.x: three-party replicated secret sharing over $\mathbb{Z}_{2^{64}}$, semi-honest, honest majority. The compiler emits a manifest of *tapes* (a main tape plus comparison subtapes). Two backend facts recur and matter for the checker under test. First, a public reveal and a legitimate masked comparison-open are the *same* opcode: MP-SPDZ “uses a single instruction for opening” [2], and the correctness-check flag does not change this. Second, a value in one tape reaches another only through memory or through register arguments passed by `call_tape` (opcode `0xec`) and received by `call_arg` (opcode `0xed`); a direct cross-tape register reference is a compiler error.

c) *Two fault models*: We keep two fault models distinct throughout; conflating them is what makes the mutation study sound either vacuous or over-strong.

Runtime protocol fault model. At most one semi-honest corrupted party among the three; honest majority; authenticated encrypted channels; process isolation (stated, *not* enforced by the single-host test harness). This is the model a *protocol-level* privacy argument would target. Our functional and conformance tests do *not* address it; that hand-off is §VII.

Build-time mutation fault model. Faults are injected into the application `.mpc` source *and only there*. The pseudo-oracle, the checker, the policy (thresholds, expected delivery), the CI workflow, the evidence validator, and the *pinned* MP-SPDZ compiler are the *trusted test infrastructure* and are *not* mutated. This is the operator space of the mutation analysis (§IV). Were the test infrastructure itself faulty, every automated check would be trivially void; stating the model is what makes the mutant accounting meaningful.

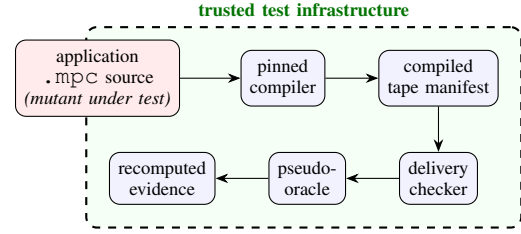


Fig. 1. Test infrastructure and the build-time mutation fault model. Mutants are injected only into the application source (red); everything in the dashed box is trusted test infrastructure. The runtime protocol fault model (not shown) is separate and is addressed only by the verification hand-off of §VII.

Fig. 1 draws the boundary. We make no claim under collusion, a malicious party, unencrypted channels, or a shared host, and no performance claim.

III. CONFORMANCE-TEST DESIGN

Each element exists to defeat a specific way a conformance test can be vacuous. We state what each element *operationally* provides, not that it is “sound.”

a) *A separately implemented pseudo-oracle*: The test oracle is a plaintext model written from the paper, separately from the circuit and from the project’s earlier exact-rational reference model, transcribing the all-possible-outputs check directly. It is a *pseudo-oracle* [3] (a separate implementation of the same specification), *not* an independently validated ground truth: it shares the specification with the circuit, so a *correlated* misreading of PLAS could make both agree on the wrong behaviour. It did carry one fatal transcription error (conditioning on the actual output only) until testing caught it. We state this shared-specification / correlated-error risk wherever we rely on oracle agreement (§VIII). This is the pseudo-oracle instance of the general oracle problem [3], and is close in spirit to metamorphic testing [4] in that it substitutes a derivable relation for an unavailable ground truth.

b) *An anti-echo interface (avoiding the oracle-in-the-SUT anti-pattern)*: Before any circuit was written, the interface was fixed so that expected outputs and expected post-state *never enter the system under test*—neither compiled in nor supplied as runtime inputs; they live only in the external harness, obtained from the pseudo-oracle. Circuit inputs are exactly the public parameters and the secret-shared secrets and beliefs, and the harness runs at least one state beyond the fixture, so a constant-output circuit cannot pass by accident. Beliefs travel as dense unnormalized non-negative integer weights; the threshold check avoids division ($bM \leq aZ$). Because MP-SPDZ comparison is signed, the no-wraparound requirement is $B < 2^{k-1}$ (not the naive $2^k > B$), which we pin with a counterexample test.

c) *A strict transcript oracle*: A correct party’s captured stdout must be exactly its two own records; any foreign, duplicate, unknown, missing, or extra line makes the test *fail closed*. This is an operational output-conformance oracle over the enumerated transcript cases on the observed stdout channel; it observes selected output channels and does not

establish absence of every possible leak. It becomes the second line of defence in §V.

d) *Recomputed, hash-bound test evidence*: Each run retains a typed record; in CI the validator requires each record’s source hash and delivery signature to equal values recomputed from the checked-out source and a fresh compile, and binds each record to a canonical case via a recomputed input hash and pseudo-oracle verdict. This establishes *self-consistency and provenance* of the test evidence *under the trusted CI workflow*; it is not integrity against an actor who can alter source, checker, workflow, and evidence together (outside the fault model of §II). It is the evidence-discipline neighbour of reproducible builds [15], recomputed per test case rather than attesting a whole artifact.

A. Test artifact and results

On the fixture (two invocations: one all-accept, one with a per-recipient reject and belief preservation) plus two further valid states, the circuit reproduces the pseudo-oracle’s visible outputs, payloads, and reconstructed weights. A *bounded, constructed* adversarial matrix of 228 cases (216 single-invocation + 12 carried; all 27 secret tuples $\times$ two queries $\times$ uniform / non-uniform / per-party-scaled integer weights near the bit bound) all match the pseudo-oracle. This is a constructed test matrix, *not* general coverage. The conformance package holds 92 collected tests—46 functional and 46 delivery-checker tests—and 9 reference test functions (25 collected items), all green; *test counts are not themselves a result*. Five committed executable negative controls are rejected by the checker (§IV). No timing is reported.

IV. MUTATION ANALYSIS OF THE DELIVERY CHECKER

Functional conformance says nothing about *how* a value is delivered: a circuit can compute correct verdicts and broadcast them. We therefore built a static delivery-conformance checker that inspects the compiled tape manifest, and evaluated its *adequacy* by mutation analysis under the build-time fault model [5], [6]. We are explicit that the first sequence is *retrospective and adaptively discovered*, not a complete or independently designed operator set: the checker was repeatedly hardened against the same mutants later used as evidence (adaptive overfitting), so this sequence yields no held-out adequacy estimate. We remove that threat in §V.

a) *Mutant accounting*: One motivating transcript failure (B0); six compiled-delivery mutants (B1–B6); one semantic mutant (S1). The six are not homogeneous: B1, B2, B3 and B5 are committed executable *source* mutants; **B4 is a synthetic compiled-manifest counterexample, not a source .mpc**; and B6 is a reconstructed source mutant whose naive realization is itself *killed* (§IV-A). B0: changing `reveal_to(j)` to a public `reveal()` while keeping the per-party print compiles to a public open of every verdict, yet a stdout-only test reported success—which motivated inspecting the *compiled* delivery. B1–B3 and B5 each *survived* the then-current checker *as committed source mutants* and were killed by adding a detection rule (separate-tape reveal; name-spoofed subtape;

`reveal(False)`—still a public open; and the vectorized-memory variant). **B4** is different: it is the observation that the multiset rule is blind to tape *bodies*—a `reveal(False)` of the verdict placed in an *existing* `EQZ(3)_63` body with the three pinned names intact—demonstrated with a *synthetic manifest*, not a compiled source mutant; against the final checker it is killed by the subtape-memory rule (the multiset left intact). The clean build stores nothing in its main tape and touches no memory in its subtapes, so forbidding memory is a restriction with no false positives on the honest build.

A. B6, at its exact strength

We scope B6 precisely, because it is easy to overstate. We reconstructed a mutant that routes party 1’s verdict through the `call_tape/call_arg` register channel (a `@function_tape` that receives the verdict as a register argument and opens it). Compiled against the pinned backend and checked by the final checker, this mutant is **KILLED**: the added tape breaks the pinned comparison-subtape multiset, and MP-SPDZ’s call frame also spills through memory, tripping the memory rules. **So B6 is not a demonstrated bypass of the final checker**; because a direct cross-tape register reference is a compile error, the B3/B5 source mutants therefore used an attacker-introduced memory hop, which the memory rules catch.

Its content is narrower and precise. The *honest* build passes secret comparison operands across the tape boundary via `call_tape` (in main) received by `call_arg` in each of the three comparison subtapes—no memory. Therefore a detection rule that forbade cross-tape register-argument transfer of secrets, the way the memory rules close the memory channel, would **reject the honest build**: the `call_arg` channel *cannot be blocklisted by opcode/channel identity* without a false positive on the honest build. Combined with the opcode-identity fact (a masked open and a raw reveal are the same instruction), the testing lesson is precise: **this opcode-identity / channel-blocklist checker cannot be promoted to a general non-leakage checker**. Distinguishing legitimate from leaking flows requires analysis stronger than opcode/channel identity—operand/provenance-sensitive dataflow, typing, a verified IR, or other semantic machinery—which we did not evaluate (§IX).

B. S1: a semantic mutant the checker cannot see

On a different axis, a mutant that drops the reject-mask (delivers the true output even to rejected recipients) **survives** every delivery-checker rule—the delivery structure is intact—yet is **killed by the pseudo-oracle**: on secrets = (0, 0, 0), `p1_is_max`, the true output is 1 and all parties reject (oracle payload 0), but S1 delivers 1 to all three. We ran this **end-to-end at the pinned backend**: all three parties complete (return code 0) and each delivers `ACCEPT 0 / PAYLOAD 1`, an executed oracle mismatch at parties [0, 1, 2] (raw evidence archived). This run is a *functional* check only; it makes no runtime, channel-security, or performance claim, and says nothing about the runtime protocol fault model. Delivery-conformance checks *how* values are delivered, not *which*;

TABLE I

MUTATION MATRIX AGAINST THE FINAL DELIVERY CHECKER (MACHINE-READABLE FORM IN THE ARTIFACT; OUTCOMES RECOMPUTED AGAINST THE FROZEN CHECKER AT REPO 305A3A8, MP-SPDZ 9D809599). “KILLED” = CHECKER REJECTS; THE FIVE COMMITTED CONTROLS MAP TO B0–B3 AND B5. B4 IS A *synthetic* COMPILED-MANIFEST COUNTEREXAMPLE, NOT A SOURCE .mpc; B6 AND S1 ARE RECONSTRUCTED SOURCE MUTANTS. NOT COMMITTED TO THE FROZEN ARTIFACT: B4, B6, S1.

ID	Committed?	Mutation / channel	Outcome	Notes
B0	leaky	public reveal() in main	KILLED	motivating stdout failure
B1	subleak	separate @function_tape reveal+print	KILLED	killed by whole-manifest inspection
B2	namespoof	name-spoofed subtape: wrong-player delivery + file sink	KILLED	killed by content rules
B3	openfalse	reveal(False) in spoofed EQZ(3)_63, scalar mem	KILLED	False open is still public
B4	<i>synthetic</i>	reveal(False) in an <i>existing</i> EQZ(3)_63 body, multiset intact	KILLED (rule h)	synthetic manifest; not a source .mpc
B5	openfalse_vec	same via vectorized vstms/vldms	KILLED	killed by broadened regex
B6	<i>reconstructed</i>	verdict via call_tape/call_arg register args	KILLED (naive)	see §IV-A; not a checker bypass
S1	<i>reconstructed</i>	correct delivery, reject-mask dropped	SURVIVES checker / KILLED by oracle	semantic mutant; checker is semantics-blind

only the functional pseudo-oracle, bounded by test coverage, constrains that.

V. A PRE-REGISTERED, HELD-OUT MUTATION STUDY

The sequence above is adaptive and cannot estimate adequacy on unseen faults (§VIII). Mutation analysis is only as credible as its independence from the tool it evaluates, so we added a *pre-registered, out-of-sample* study: a corpus of 22 mutants with locked predictions was committed *before* execution (pre-registration) and run once against the frozen checker. It is out-of-sample but *not* an independent-adversary evaluation—the mutants and predictions were authored within the same project that built the checker—so we report per-category outcomes and *no* aggregate mutation score.

A. Mutation operators and the pre-registration protocol

The 22 held-out mutants instantiate seven operator categories, drawn from three sources so the corpus is not a renaming of B0–B6: security mutation-testing practice (systematic operator families, and separating *stillborn* and *equivalent* mutants from killed ones [6], [7]); the information-flow observation that leakage is a dataflow rather than opcode-identity property, which *predicts* surviving value-routed and permutation mutants (§IX); and MP-SPDZ back-end mechanisms (the open family, the `call_tape/call_arg` register channel, memory forms, sink opcodes, and the cross-tape compile error). The categories are: (1) recipient/destination; (2) public/private open; (3) subtape/topology; (4) secret-transfer/channel; (5) output/value-semantic; (6) transcript/output-channel; and (7) evidence/provenance (which targets the *separate* evidence validator, not the delivery checker).

The protocol is two-phase and its integrity is visible in the commit history. First, the corpus, the mutant sources or synthetic transforms, and a *locked prediction* for every mutant—

predicted compile outcome, predicted checker verdict, source realizability, and denominator membership—are committed *before* the checker is run on them. Second, the study is executed *once* and the machine-readable results are committed separately. We then classify each mutant honestly: *killed* (checker rejects), *survived* (checker accepts), *stillborn* (does not compile; never counted as a kill), and *benign/equivalent* (correctly not flagged). We record per mutant whether it is *source-realizable* (a compiled .mpc) or a *synthetic manifest-level probe* whose source realizability is unproven, and we report **no aggregate mutation score over the synthetic probes**: a surviving probe shows only that no rule fires on that assembly pattern, not that a source program can produce it (B6 shows the naive realization is killed). All 22 outcomes matched their pre-registered predictions—evidence about the checker’s *rules*, not about an independent adversary’s ingenuity (§VIII).

The decisive result concerns delivery *structure*. Among three pre-registered source-realizable recipient-structure mutants, the checker killed H-R1 and H-R3 and *survived* the multiset-preserving H-R2 permutation. **H-R2 is a demonstrated source-realizable structural fault the checker misses**: a compiled build that delivers each party’s verdict to the *wrong* recipient while preserving the pinned destination-player multiset [0, 0, 1, 1, 2, 2]—all that the main-tape delivery rule inspects. It is killed only downstream: the strict transcript oracle (§III) rejected *this concrete* build because a foreign `PRIV.j` record appears on the receiving party’s stream; we do *not* claim the transcript oracle catches recipient permutations in general. Three source-realizable *semantic* mutants (negated accept, swapped accept/payload, a neighbour’s payload) likewise survive the checker and are killed by the pseudo-oracle, echoing S1. The remaining *synthetic* mutants are *manifest-level rule-coverage probes*—hand-built assembly patterns testing whether a rule fires, with *unproven* source realizability,

not executable mutants, and excluded from any mutation-score denominator (the corpus also contains provenance and stillborn/compile-invalid cases, scored separately). The informative probe is H-O3: a `call_arg` open of a register-passed value in an existing masked subtape that no current rule rejects—the same gap B6 names (§IV-A), now exhibited at the manifest level. Table II summarizes the outcome classes.

VI. REPRODUCTION AND CROSS-VERSION REPLICATION

The frozen artifact reproduces from a fresh checkout with a fresh build of the pinned MP-SPDZ backend: 117 collected `pytest` items (the 92 conformance items plus the 9 reference test functions, which expand to 25), the named-case harness 4/4, the 228-case matrix 228/228, the recomputed evidence, and the B4/B6/S1 reproductions all reproduce, and the normalized per-tape `manifest_signature` values recompute to the archived values. We reproduce those *normalized-assembly* signatures, *not* raw binaries or assembly byte-for-byte. Separately, the backend-level test observations replicate on one further MP-SPDZ version, v0.4.3 (26a60536): the shared open-instruction family, the honest `call_tape/call_arg` path, the B6 behaviour, and S1’s structural survival with an executed oracle mismatch all recur, with no compile/API or semantic change in what we exercised. This is two versions, not a cross-version compatibility or version-independence claim: only one additional version, on one host, for the probed observations, was tested. A one-command reproduction wrapper and a machine-readable results file are in the artifact.

VII. WHAT THE TEST SUITE ESTABLISHES

Separating three levels keeps the claims honest.

- 1) **General prior principle (not novel here):** information-flow safety depends on dataflow and semantics, not opcode names—established for constant-time and smart-contract bytecode (§IX).
- 2) **Case-study lesson (transferable, not empirically validated across applications):** adaptive checking of a security property can produce false confidence, and such tooling should print its adequacy boundary; ours does.
- 3) **Backend finding (backend-specific; observed at the pin 9d809599 and one additional version, v0.4.3, not version-independent):** MP-SPDZ’s shared open instruction and the `call_tape/call_arg` path, and the consequence that the studied opcode-identity / channel-blocklist checker cannot be promoted to a general non-leakage checker.

a) A layered account: Concretely the assurance is a chain of partial checks, no one of them a privacy proof. The pinned *compiler* rejects one naive cross-tape transfer outright (a direct cross-tape register reference is a compile error; the B3/B5 source mutants therefore used an attacker-introduced memory hop). The *delivery checker* kills gross structural and channel mutant classes—public opens, illicit sinks, disallowed memory, off-pin tapes—including forms outside its design set (exercised as manifest-level probes, not executable source mutants), yet H-R2 shows it survives a multiset-preserving

wrong-recipient *structure*. The *transcript oracle* killed that one concrete mutant; the *pseudo-oracle* kills semantic mutants delivery-checking survives (S1 and the held-out semantic mutants); and *evidence recomputation* protects the consistency and provenance of these records under the trusted CI. In the tested examples, later layers catch some faults earlier checks leave open; the layers are complementary, not complete, and none, alone or together, is the protocol-level privacy proof the hand-off below still requires.

b) What would make it a guarantee: Two verification obligations, neither supplied by our tests: (i) *privacy of the executed circuit*—a Rep3 simulation argument or a formally-verified, dataflow-checked comparison primitive; and (ii) *semantic source-to-spec binding*—that the circuit computes the intended function, not merely one that agrees with the pseudo-oracle on tested cases (S1 shows why delivery privacy is insufficient without it). A Rep3 proof establishes (i) and not (ii); our tests give coverage-bounded evidence toward (ii) and nothing toward (i).

VIII. THREATS TO VALIDITY

- **Construct validity (test infrastructure / fault model).** All results assume the trusted test infrastructure of §II; an actor that also mutates the oracle, checker, policy, CI, or compiler voids them. Workflow hashes establish *self-consistency*, not authenticity against such an actor.
- **Correlated authorship and AI assistance.** Circuit, pseudo-oracle, tests, and manuscript were produced within one project and substantially with AI assistance; model agreement is not independent validation.
- **Adaptive overfitting, partly mitigated.** The checker was hardened against the same mutants later used as evidence. We added a *pre-registered, out-of-sample* corpus run once against the frozen checker (§V); it is out-of-sample but *not* independent-adversary (same-project authorship), so we report per-category outcomes rather than a mutation score.
- **Pseudo-oracle pseudo-independence.** The oracle shares PLAS with the circuit; a correlated spec misreading is possible. A cross-check against the project’s exact reference model is limited because that model implements a *different* (single-observer, stateless) functionality; we state the risk rather than claim it eliminated.
- **External validity.** “Conformance” means agreement with a pseudo-oracle on tested inputs. Results are for one deterministic-query functionality, $N=3$, $D=\{0, 1, 2\}$, one backend, and a single-host harness that is not an adversarially isolated deployment; the backend-level observations were additionally checked on one further MP-SPDZ version (v0.4.3), which is two versions, not a cross-version compatibility claim. We do not imply an evaluated methodology across MPC applications plural.
- **No performance or simulation-security claim;** the boundary result is a case-study observation, not a theorem.

TABLE II

HELD-OUT MUTANT OUTCOME CLASSES (22 MUTANTS, SINGLE RUN AGAINST THE FROZEN CHECKER; PREDICTIONS PRE-REGISTERED BEFORE EXECUTION). “SURVIVED” = CHECKER PASSES. NO AGGREGATE MUTATION SCORE IS REPORTED FOR THE MANIFEST-LEVEL PROBES, WHOSE SOURCE REALIZABILITY IS UNPROVEN.

Class	Mutants	Outcome
source-realizable, recipient	H-R1, H-R3	killed by checker
source-realizable, recipient	H-R2	survives checker; killed by transcript oracle (this mutant)
source-realizable, semantic	H-V1–H-V3	survive checker; killed by pseudo-oracle
synthetic manifest probe	H-O1,H-O2,H-T2,H-T3,H-C1,H-C2,H-P1–H-P3	rule fires (probe “killed”)
synthetic manifest probe	H-O3	no rule fires (<i>survives</i>); realizability unproven
synthetic (benign)	H-T1, H-C3	correctly not flagged
stillborn (compile-invalid)	H-X1, H-X2	do not compile; not a kill
provenance (other checker)	H-E1, H-E2	rejected by the evidence validator

IX. RELATED WORK

Novelty statements below rest on a bounded, documented search; they are “to the best of a bounded search,” not absolute (§VIII).

a) Test oracles and pseudo-oracles: The oracle problem—deciding the expected result of a test—is surveyed by Barr et al. [3]; a pseudo-oracle (a second implementation of the same specification) and metamorphic testing [4] are the standard responses when no ground truth exists. Our anti-echo interface is a discipline for avoiding the oracle-in-the-SUT trap in this setting; Wycheproof applies reference-oracle conformance to crypto *primitives* by feeding expected outputs, the opposite of our anti-echo design.

b) Mutation testing and negative controls: Mutation testing dates to DeMillo et al. [5] and is surveyed by Jia and Harman [6]; whether mutants substitute for real faults is studied by Just et al. [7]. We use mutants as an *adequacy probe for a checker* and are careful about mutant classes—killed, survived, stillborn (compile-invalid), and benign/equivalent—never counting a stillborn mutant as a kill. Closest in spirit, MASC [8] evaluates crypto-misuse detectors with planted leaky mutants, but as an offline third-party-detector benchmark rather than committed CI controls; and, unlike a standard mutation-score study, we *pre-register* a held-out corpus to counter the adaptive coupling between a checker and the mutants used to harden it.

c) Leakage is dataflow, not opcode identity: ct-verif reduces constant-time to a 2-safety property on LLVM IR [9]; Binsec/Rel decides constant-time *relationally at the binary/opcode level* on operand provenance, showing compiler backends inject leaks invisible to instruction-name checks [10]; Securify decides EVM-bytecode security on inferred *dataflow* facts rather than syntactic patterns [11]. Our §IV-A instantiates this principle in the MPC-application testing setting; we claim the instantiation and the pinned-backend mechanism, not the principle.

d) Language-based security for MPC (the positive dual): Kerschbaum’s information-flow type system certifies non-leakage of a mixed-protocol secure computation at the *source* level with a trusted compiler [12]. Closest to our finding, Skalka and Near give a *sound* static security-type system

for a purpose-built low-level MPC IR, proving information is released only through explicit declassifications [13], [14]. It is the positive dual of our negative case-study observation: it is sound *because* its IR makes declassification syntactically distinct—the property MP-SPDZ bytecode lacks. Reproducible Builds [15] is the evidence-discipline neighbour; ours recomputes per-case, oracle-bound test evidence rather than attesting whole artifacts.

X. CONCLUSION

On one MP-SPDZ application, a separately implemented pseudo-oracle, an anti-echo interface, a strict transcript oracle, and recomputed evidence give real, coverage-bounded, operationally-stated test assurance and kill a family of gross delivery-fault classes—and they have a sharp, nameable adequacy boundary. A static opcode-identity / channel-blocklist checker cannot be promoted to a general non-leakage checker: a masked open and a raw reveal are the same instruction, and the register channel a leak would use is the one the honest code uses. A pre-registered, held-out mutation study makes the boundary concrete with a *surviving* source-realizable mutant (H-R2, a multiset-preserving mis-delivery) caught only by a downstream oracle. Distinguishing the flows needs analysis stronger than opcode/channel identity—which we did not evaluate—and a privacy guarantee needs a protocol-level proof. Naming that boundary—and printing it next to the green check—is the case study’s point.

ARTIFACT AVAILABILITY

The system under test, the pseudo-oracle, the frozen delivery checker (`delivery_inspect.py`), the five committed negative controls, the mutation corpus and machine-readable mutation matrix, the pre-registered held-out corpus with its locked predictions and single-run results, and one-command reproduction wrappers are provided as *anonymized supplementary material* and will be released. The conformance artifact is frozen at a single commit; the held-out study’s pre-registration and results are separate, time-ordered commits. All quantitative claims in this paper reproduce from a fresh checkout with a fresh build of the pinned MP-SPDZ backend.

REFERENCES

- [1] P. Mardziel, M. Hicks, J. Katz, M. Srivatsa. Knowledge-Oriented Secure Multiparty Computation. PLAS, 2012.
- [2] M. Keller. MP-SPDZ: A Versatile Framework for Multi-Party Computation. ACM CCS, 2020, pp. 1575–1590. IACR ePrint 2020/521.
- [3] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, S. Yoo. The Oracle Problem in Software Testing: A Survey. IEEE TSE, 41(5), 2015, pp. 507–525.
- [4] T. Y. Chen, F.-C. Kuo, H. Liu, P.-L. Poon, D. Towey, T. H. Tse, Z. Q. Zhou. Metamorphic Testing: A Review of Challenges and Opportunities. ACM Computing Surveys, 51(1), 2018.
- [5] R. A. DeMillo, R. J. Lipton, F. G. Sayward. Hints on Test Data Selection: Help for the Practicing Programmer. IEEE Computer, 11(4), 1978, pp. 34–41.
- [6] Y. Jia, M. Harman. An Analysis and Survey of the Development of Mutation Testing. IEEE TSE, 37(5), 2011, pp. 649–678.
- [7] R. Just, D. Jalali, L. Inozemtseva, M. D. Ernst, R. Holmes, G. Fraser. Are Mutants a Valid Substitute for Real Faults in Software Testing? ACM FSE, 2014, pp. 654–665.
- [8] A. S. Ami, N. Cooper, K. Kafle, K. Moran, D. Poshyvanyk, A. Nadkarni. Why Crypto-detectors Fail: A Systematic Evaluation of Cryptographic Misuse Detection Techniques. IEEE S&P, 2022.
- [9] J. B. Almeida, M. Barbosa, G. Barthe, F. Dupressoir, M. Emmi. Verifying Constant-Time Implementations. USENIX Security, 2016.
- [10] L.-A. Daniel, S. Bardin, T. Rezk. Binsec/Rel: Efficient Relational Symbolic Execution for Constant-Time at Binary Level. IEEE S&P, 2020.
- [11] P. Tsankov et al. Securify: Practical Security Analysis of Smart Contracts. ACM CCS, 2018.
- [12] F. Kerschbaum. An Information-Flow Type-System for Mixed Protocol Secure Computation. ACM ASIACCS, 2013.
- [13] C. Skalka, J. P. Near. Language-Based Security for Low-Level MPC. PPDP, 2024. arXiv:2407.16504.
- [14] C. Skalka, J. P. Near. SMT-Boosted Security Types for Low-Level MPC. ESOP 2025, LNCS 15695, pp. 258–285. arXiv:2501.17824.
- [15] C. Lamb, S. Zacchiroli. Reproducible Builds: Increasing the Integrity of Software Supply Chains. IEEE Software, 2022.